深度学习求解微分方程：算法、理论、应用 I

程青

同济大学

2026.05.12，上海创智学院



Contents

- Introduction
  - Classical numerical methods
  - Deep neural networks
- Solving PDEs using deep learning
  - PINNs
  - Deep Ritz method
  - Approximation theory
- Numerical examples
  - Some simple examples
  - Adaptive variants
  - 3D shock waves

Classical Numerical methods

- Finite difference method (FDM)
- Finite element method (FEM)
- Finite volume method (FVM)
- Spectral method (SM)
- *h*-*p* method
- ……

Drawback of Classical Numerical discretization for Solving PDEs

- ✓ Mesh generation remains complex
- ✓ **High-dimensional problems** cannot be tackled (**Curse of dimensionality**)
- ✓ Solving **inverse problems** is often prohibitively expensive
- ✓ Cannot seamlessly incorporate **noisy data**
- ✓ Not flexible

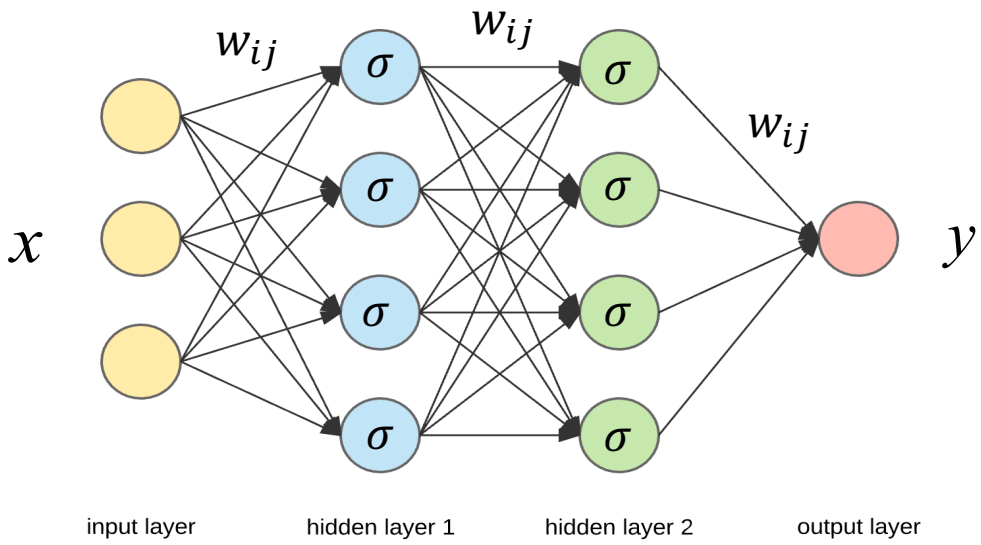


New opportunity: Machine learning

Contents

- Introduction
  - Classical numerical methods
  - Deep neural networks
- Solving PDEs using deep learning
  - PINNs
  - Deep Ritz method
  - Approximation theory
- Numerical examples
  - Some simple examples
  - Adaptive variants
  - 3D shock waves

Deep Neural Networks (DNNs)



- $x \in R^{d_x} \rightarrow NN \rightarrow y \in R^{d_y}$
- Width: N_l
- Depth: # of hidden layers + 1
- σ: activation function

- Input: $x \in R^{d_x}$
- Output of 1st hidden layer: $y_1 = \sigma(W_1^T x + b_1), W_1 \sim d_x \times N_1$
- Output of 2nd hidden layer: $y_2 = \sigma(W_2^T y_1 + b_2), W_2 \sim N_1 \times N_2$
- ...
- Output of last layer: $y_l = W_l^T y_{l-1} + b_l$

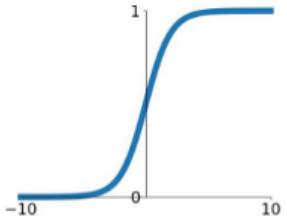
$$u_{\mathrm{N}}(\mathrm{x}) = \sum_{k=1}^{N} \hat{u}_k \, \phi_k(x)$$

DNNs: About activation functions

Activation Functions

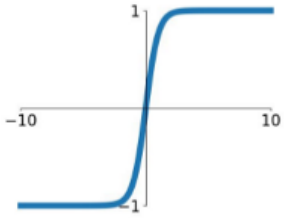
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



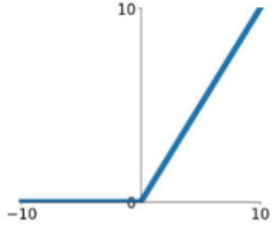
tanh

$$\tanh(x)$$



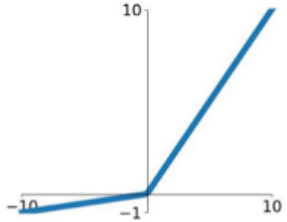
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

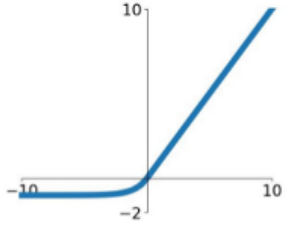


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Remarks:

- First choice: tanh
- For very deep NNs, try ReLu-type activation function first

DNNs: About activation functions

东方理工高等研究院
EASTERN INSTITUTE FOR ADVANCED STUDY
宁波东方理工大学(暂名)

Sigmoid



$$y = \frac{1}{1+e^{-x}}$$

Tanh



$$y = \tanh(x)$$

Step Function



$$y = \begin{cases} 0, & x<n \\ 1, & x \geq n \end{cases}$$

Softplus



$$y = \ln(1+e^{x})$$

ReLU



$$y = \begin{cases} 0, & x<0 \\ x, & x \geq 0 \end{cases}$$

Softsign



$$y = \frac{x}{(1+|x|)}$$

ELU



$$y = \begin{cases} \alpha(e^{x}-1), & x<0 \\ x, & x \geq 0 \end{cases}$$

Log of Sigmoid



$$y = \ln\left(\frac{1}{1+e^{-x}}\right)$$

Swish



$$y = \frac{x}{1+e^{-x}}$$

Sinc



$$y = \frac{\sin(x)}{x}$$

Leaky ReLU



$$y = \max(0.1x, x)$$

Mish



$$y = x(\tanh(\text{softplus}(x)))$$

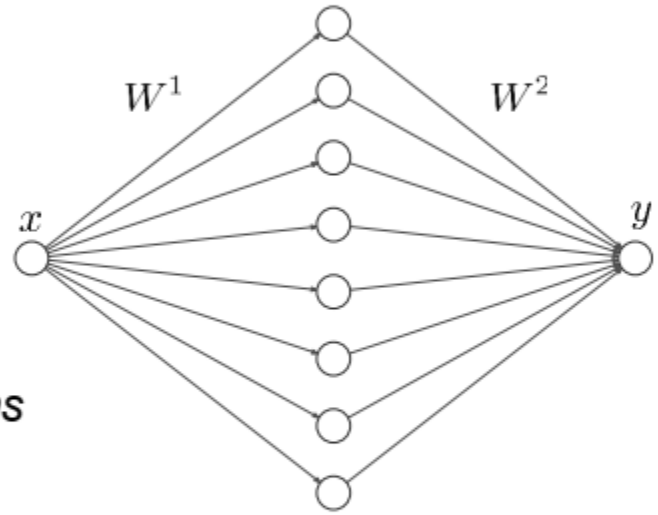
Universal Function Approximation (single layer)

Definition. We say that σ is *discriminatory* if for a measure $\mu \in M(I_N)$

$$\int_{I_N} \sigma(W^1 x + b^1) d\mu(x) = 0$$

for all $W^1 \in \mathbb{R}^n$ and $b^1 \in \mathbb{R}^n$ implies that $\mu = 0$

The space of finite, signed regular Borel measures on I_N is denoted by $M(I_N)$



Definition. We say that σ is *sigmoidal* if

$$\sigma(x) \to \begin{cases} 1 & \text{as } x \to +\infty \\ 0 & \text{as } x \to -\infty \end{cases}$$

Theorem 1. Let σ be *any continuous discriminatory function. Then finite sums of the form*

$$y = \sum_{j=1}^{N} W_j^2 \sigma(W_j^1 x + b_j^1)$$

are dense in $C(I_N)$. *In other words, given any* $f \in C(I_N)$ *and* $\epsilon > 0$, *there is a sum,* y, *of the above form, for which*

$$|y - f(x)| < \epsilon \qquad for\ all\ x \in I_N$$

➢ **Note**: The set of all functions y does not form a vector space since it is not closed under addition.

G. Cybenko, "Approximation by superpositions of a sigmoidal function", Mathematics of Control, Signals and Systems, 303-314, 2(4), 1989

Universal Functional Approximation (single layer)

Theorem (*Chen and Chen, 1993*):

Suppose that U is a compact set in $C[a,b], f$ is a continuous functional defined on U, and $\sigma(x)$ is a bounded sigmoidal function, then for any $\epsilon > 0$, there exist $m+1$ points $a = x_0 < ... < x_m = b$, a positive integer N and constants $W_i^2, b_i, W_{i,j}, i = 1, 2, ..., N,$ $j = 1, 2, ..., m$

Such that

$$\left| f(u) - \sum_{i=1}^{N} W_i^2 \sigma \left(\sum_{j=0}^{m} W_{i,j}^1 u(x_j) + b_i \right) \right| < \epsilon$$

holds for all $u \in U$.

T.P. Chen and H. Chen, Approximations of continuous functionals by neural networks with application to dynamic systems, IEEE Transactions on Neural Networks, 910-918, 4(6), 1993.

Contents

- Introduction
  - Classical numerical methods
  - Deep neural networks
- Solving PDEs using deep learning
  - PINNs
  - Deep Ritz method
  - Approximation theory
- Numerical examples
  - Some simple examples
  - Adaptive variants
  - 3D shock waves

Machine Learning for PDEs

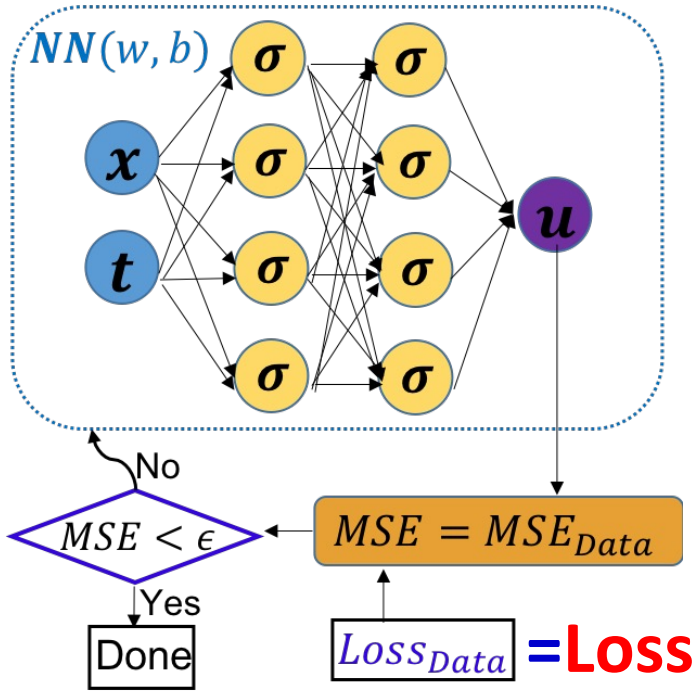
东方理工高等研究院 EASTERN INSTITUTE FOR ADVANCED STUDY 宁波东方理工大学(暂名)

- ✓ Machine learning based on *Gauss Process* (E. Brochu, et al, 2010)
- ✓ Neural network-based methods for ODEs and PDEs (I. E. Lagaris, A. Likas and D. I. Fotiadis, *IEEE Trans. Neural Netw*, 1998)
- ✓ SINdy (S. H. Rudy et al, *Science Advances*, 2017):
- ✓ Physics-informed Neural Networks (PINNs, M. Raissi, P. Perdikaris, G. E. Karniadakis, *JCP*, 2019).
- ✓ Deep Ritz method (W. E, B. Yu, *CMS*, 2018)
- ✓ Deep Galerkin Method (J. Sirignano, K. Spiliopoulos, *JCP*, 2018)
- ✓ WAN (Gao et al, *JCP*, 2020).

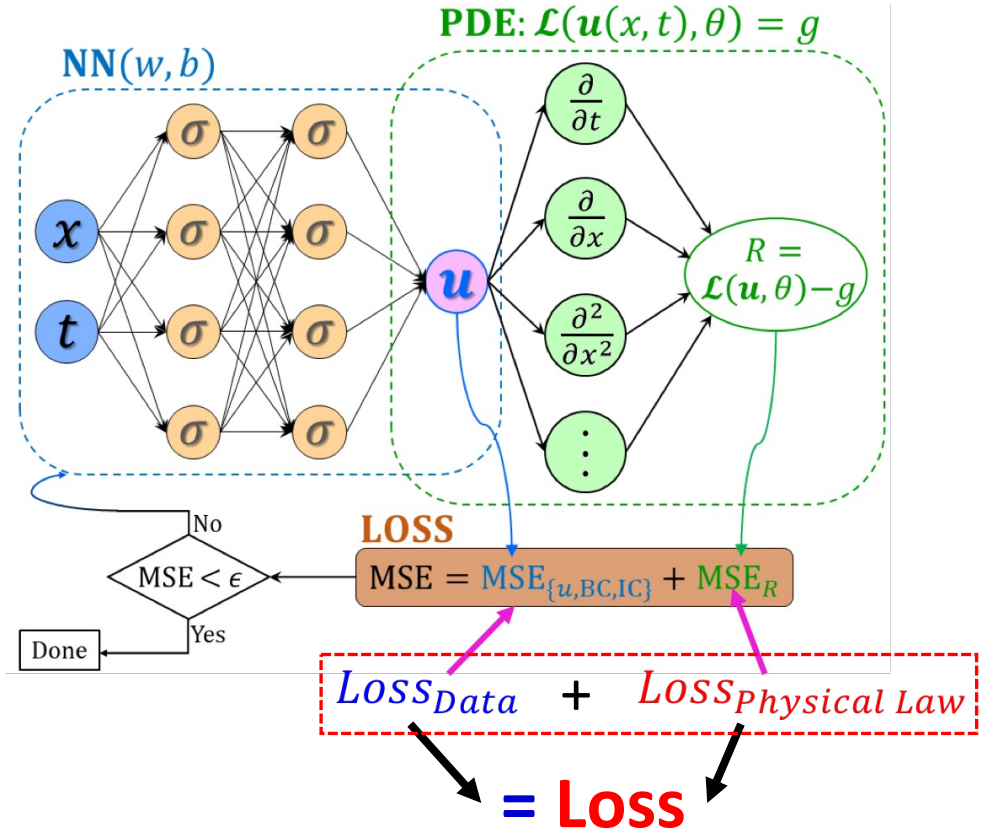
Physical-Informed Neural Network (PINN)

东方理工高等研究院 EASTERN INSTITUTE FOR ADVANCED STUDY
宁波东方理工大学（暂名）

❑ Data driven model



❑ PINN (Raissi, Perdikaris and Karniadakis, 2019, JCP)



- $a^l = \sigma\left(w^l a^{l-1} + b^l, l = 1, \cdots, L-1\right), \quad u^L = w^L a^{L-1} + b^L,$
- σ: activation function, for example, tanh, ReLU, adaptive activation functions (Jagtap, Kawaguchi and Karniadakis, 2020, JCP) ……

Adaptive tanh: tanh(ax)
Adaptive ReLU: max(0, ax)

PINNs: Forward problem (λ is known)

Consider:

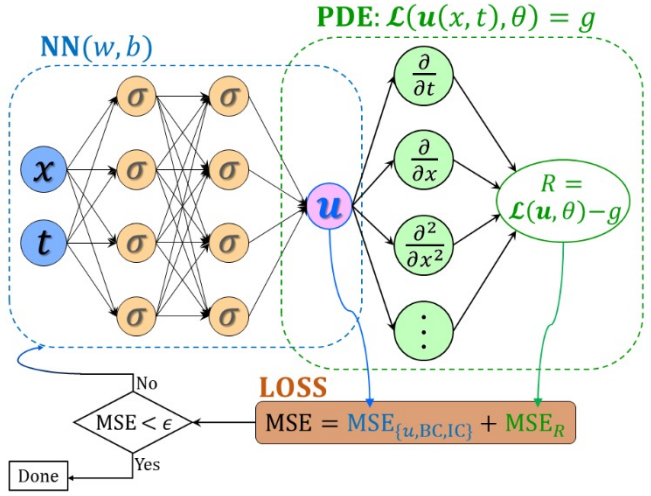
$$f\left(\mathbf{x}; \frac{\partial u}{\partial x_1}, \ldots, \frac{\partial u}{\partial x_d}; \frac{\partial^2 u}{\partial x_1 \partial x_1}, \ldots, \frac{\partial^2 u}{\partial x_1 \partial x_d}; \ldots; \boldsymbol{\lambda}\right) = 0, \quad \mathbf{x} \in \Omega$$

$$\mathcal{B}(u, \mathbf{x}) = 0 \quad \text{on} \quad \partial\Omega$$

The NN solution is $u_{\mathrm{N}}(\mathrm{x}) = \sum_{k=1}^{N} a_k \, \phi_k(x; \theta)$

Loss function for the forward problem

$$\mathcal{L}(\boldsymbol{\theta}; \mathcal{T}) = w_f \mathcal{L}_f(\boldsymbol{\theta}; \mathcal{T}_f) + w_b \mathcal{L}_b(\boldsymbol{\theta}; \mathcal{T}_b)$$



where

$$\mathcal{L}_f(\boldsymbol{\theta}; \mathcal{T}_f) = \frac{1}{|\mathcal{T}_f|} \sum_{\mathbf{x} \in \mathcal{T}_f} \left\| f\left(\mathbf{x}; \frac{\partial \hat{u}}{\partial x_1}, \ldots, \frac{\partial \hat{u}}{\partial x_d}; \frac{\partial^2 \hat{u}}{\partial x_1 \partial x_1}, \ldots, \frac{\partial^2 \hat{u}}{\partial x_1 \partial x_d}; \ldots; \boldsymbol{\lambda}\right) \right\|_2^2,$$

$$\mathcal{L}_b(\boldsymbol{\theta}; \mathcal{T}_b) = \frac{1}{|\mathcal{T}_b|} \sum_{\mathbf{x} \in \mathcal{T}_b} \|\mathcal{B}(\hat{u}, \mathbf{x})\|_2^2,$$

Forward problem

1. Construct a neural network $\hat{u}(\mathbf{x};\boldsymbol{\theta})$ with parameters $\boldsymbol{\theta}$.
2. Specify the two training sets $\mathcal{T}_f$ and $\mathcal{T}_b$ for the equation and IC/BCs.
3. Specify a loss function of both the PDE equation and IC/BC residuals

$$\mathcal{L}(\boldsymbol{\theta};\mathcal{T}) = w_f\mathcal{L}_f(\boldsymbol{\theta};\mathcal{T}_f) + w_b\mathcal{L}_b(\boldsymbol{\theta};\mathcal{T}_b)$$

where

$$\mathcal{L}_f(\boldsymbol{\theta};\mathcal{T}_f) = \frac{1}{|\mathcal{T}_f|}\sum_{\mathbf{x}\in\mathcal{T}_f}\left\| f\left(\mathbf{x}; \frac{\partial\hat{u}}{\partial x_1},\ldots; \frac{\partial^2\hat{u}}{\partial x_1\partial x_1},\ldots;\ldots;\boldsymbol{\lambda}\right)\right\|_2^2$$

$$\mathcal{L}_b(\boldsymbol{\theta};\mathcal{T}_b) = \frac{1}{|\mathcal{T}_b|}\sum_{\mathbf{x}\in\mathcal{T}_b}\|\mathcal{B}(\hat{u},\mathbf{x})\|_2^2$$

4. Train the neural network to find the best parameters $\boldsymbol{\theta}^*$ by minimizing the loss function $\mathcal{L}(\boldsymbol{\theta};\mathcal{T})$.

Solving the heat equation using PINNs

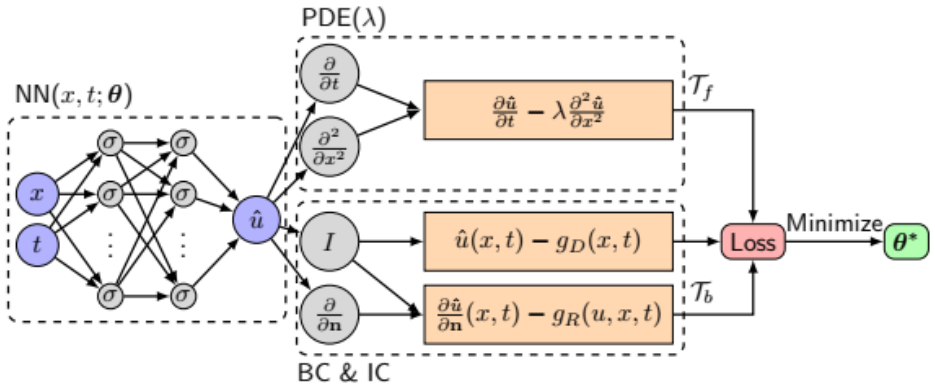


Figure: Schematic of a PINN for solving $\frac{\partial u}{\partial t} = \lambda \frac{\partial^2 u}{\partial x^2}$. The initial condition is treated as a special type of BCs. $\mathcal{T}_f$ and $\mathcal{T}_b$ denote the two sets of residual points for the equation and BC/IC.

- We define the loss function

$$\mathcal{L}(\boldsymbol{\theta};\lambda) = \omega_f \mathcal{L}_f(\boldsymbol{\theta};\lambda) + \omega_b \mathcal{L}_b(\boldsymbol{\theta}),$$

  where $\mathcal{L}_f(\boldsymbol{\theta};\lambda) = \frac{1}{|\mathcal{T}_f|} \sum_{x \in \mathcal{T}_f} \left\| \frac{\partial \hat{u}}{\partial t} - \lambda \frac{\partial^2 \hat{u}}{\partial x^2} \right\|_2^2$, $\mathcal{L}_b(\boldsymbol{\theta}) = \frac{1}{|\mathcal{T}_b|} \sum_{x \in \mathcal{T}_b} \|\mathcal{B}(\hat{u}, x)\|_2^2$.

- Automatic differentiation (tf.gradient).

Comparison between PINN and FEM

Table: Comparison between PINN and FEM.

	PINN	FEM
Basis function	Neural network (nonlinear)	Piecewise polynomial (linear)
Parameters	Weights and biases	Point values
Training points	Scattered points (mesh-free)	Mesh points
PDE embedding	Loss function	Algebraic system
Parameter solver	Gradient-based optimizer	Linear solver

Contents

- Introduction
  - Classical numerical methods
  - Deep neural networks
- Solving PDEs using deep learning
  - PINNs
  - Deep Ritz method
  - Approximation theory
- Numerical examples
  - Some simple examples
  - Adaptive variants
  - 3D shock waves

Deep Ritz Method (E and Yu, Commun. Math. Stat. 2018)

东方理工高等研究院
EASTERN INSTITUTE FOR ADVANCED STUDY
宁波东方理工大学(暂名)

- Consider the Poisson equation

$$\begin{cases} -\Delta u = f, x \in \Omega, \\ \text{BCs} \end{cases}$$

- The corresponding variational problem is

$$u = \min_{w \in H} I[w]$$

where

$$I[w] = \int_{\Omega} \left(\frac{1}{2} |\nabla w|^2 - f w \right) dx$$

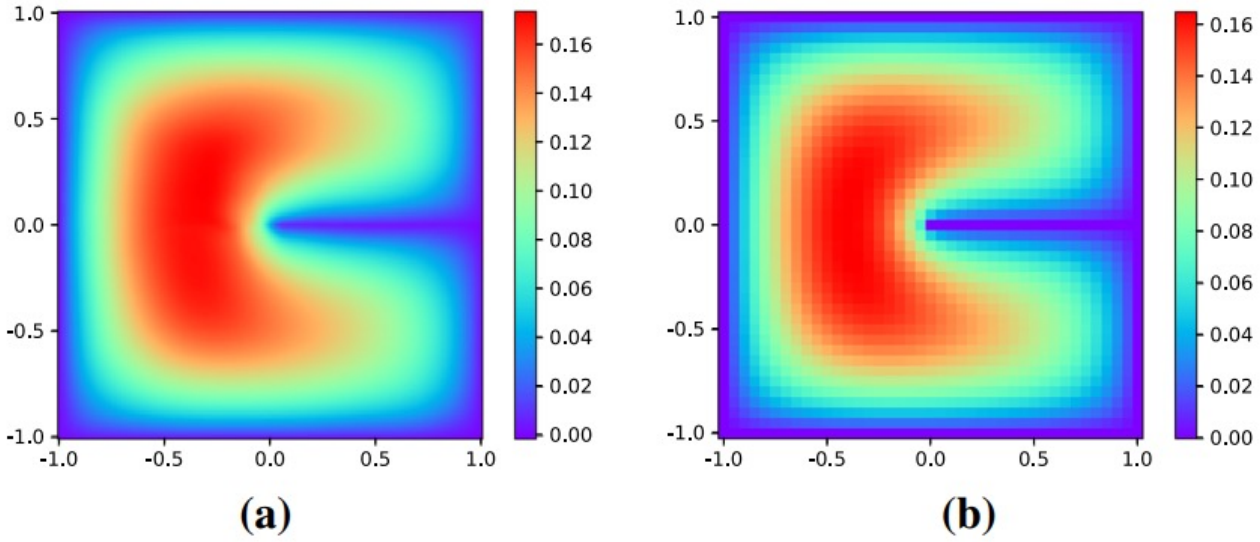
Deep Ritz Method (E and Yu, Commun. Math. Stat. 2018)

东方理工高等研究院 EASTERN INSTITUTE FOR ADVANCED STUDY
宁波东方理工大学(暂名)

➢ Loss function

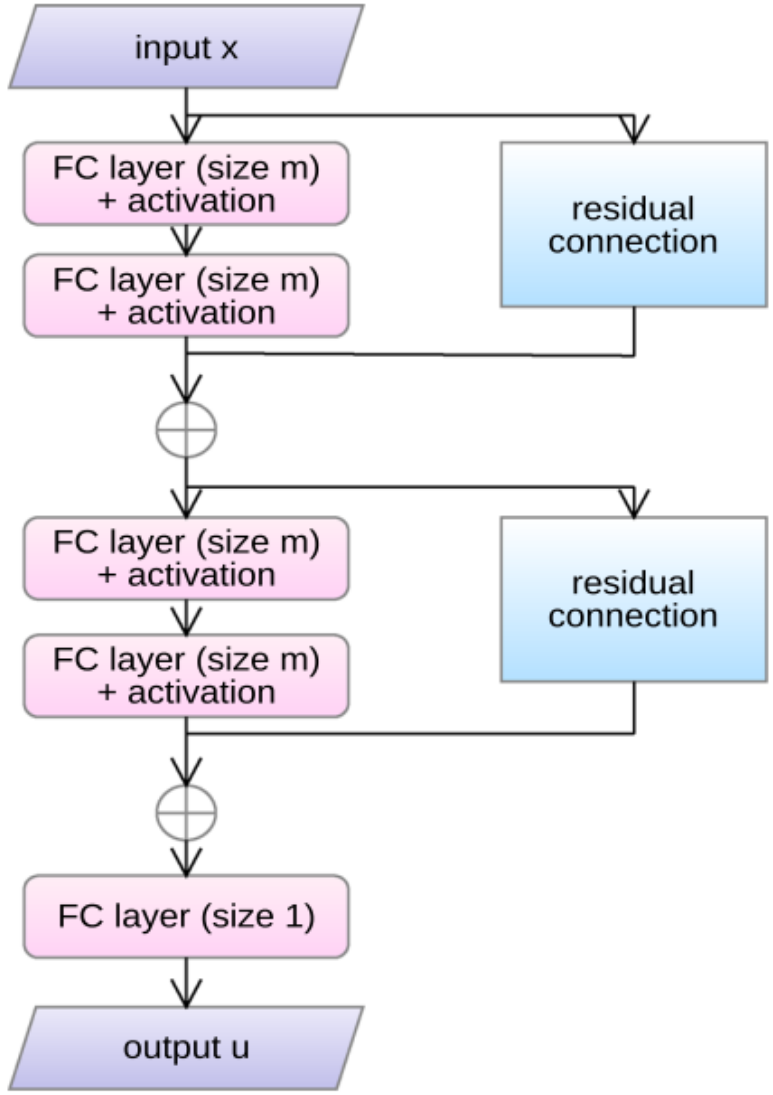
$$g(x;\theta) = \frac{1}{2}|\nabla_x u(x;\theta)|^2 - f(x)u(x;\theta),$$

$$\min_{\theta} L(\theta), \quad L(\theta) = \int_{\Omega} g(x;\theta)\mathrm{d}x.$$



Deep Ritz method

Finite difference method

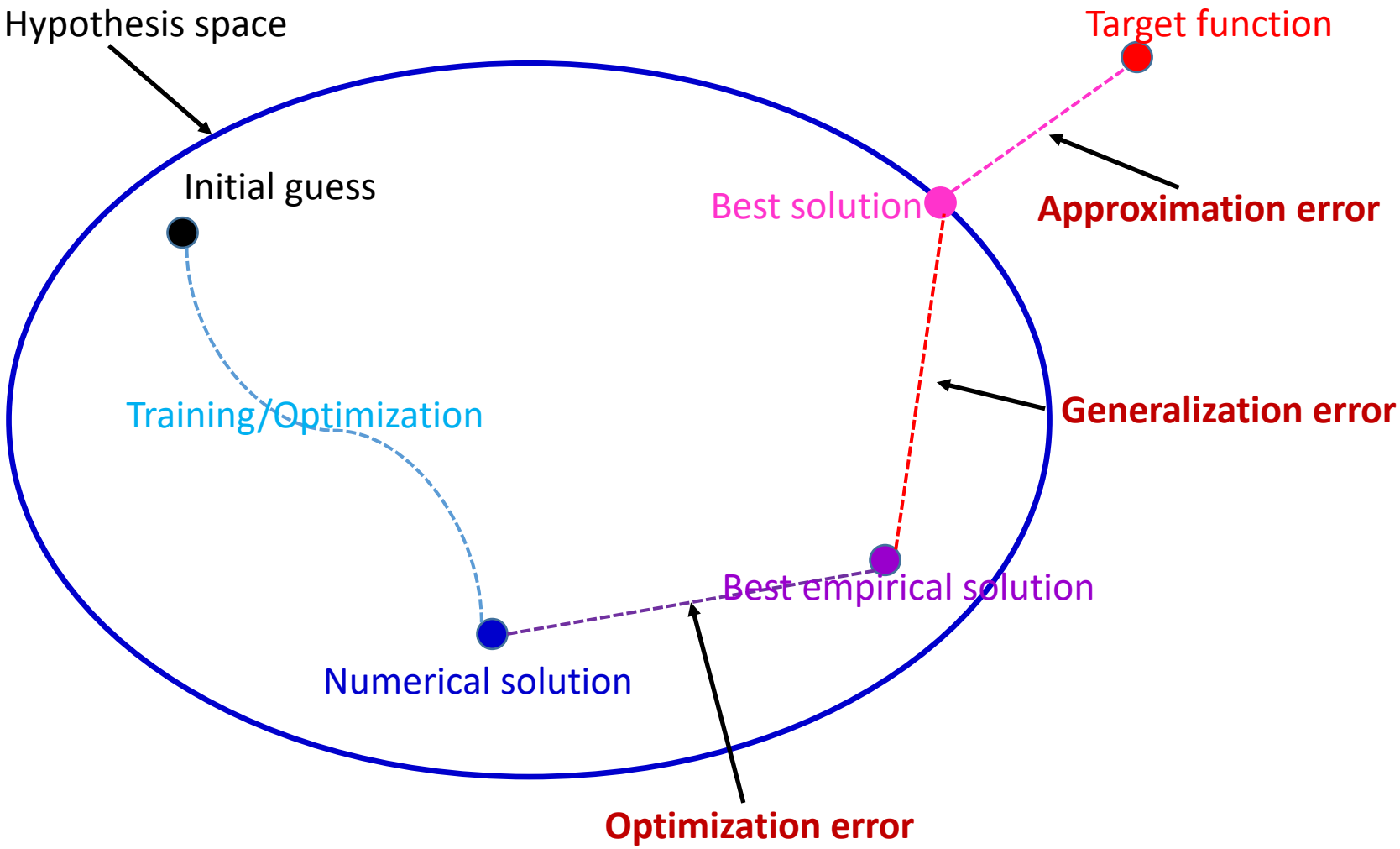


Contents

- Introduction
  - Classical numerical methods
  - Deep neural networks
- Solving PDEs using deep learning
  - PINNs
  - Deep Ritz method
  - Approximation theory
- Numerical examples
  - Some simple examples
  - Adaptive variants
  - 3D shock waves

Error analysis of Numerical approximation

东方理工高等研究院
EASTERN INSTITUTE FOR ADVANCED STUDY
宁波东方理工大学(暂名)



Classical approximation estimate

东方理工高等研究院
EASTERN INSTITUTE FOR ADVANCED STUDY
宁波东方理工大学(暂名)

- A priori error estimates

$$\|f - f_n\|_X \leq C n^{-\alpha(d)} \|f\|_Y$$

- A posteriori error estimates

$$\|f - f_n\|_X \leq C n^{-\beta(d)} \|f_n\|_Z$$

- **Curse of Dimensionality! Here $\alpha(d), \beta(d)$ depends on the dimension d.**

Fourier approximation

- Approximation estimate

$$\|f - f_{\widetilde{N}}\| \leq C\widetilde{N}^{-\frac{m}{d}}\|f\|_{H^m_{per}(\Omega)},$$

where d is the dimension, $\widetilde{N}$ is the total degree of freedom, $H^m_{per}(\Omega)$ is the Sobolev space.

Legendre polynomial approximation

- Approximation estimate

$$\|f - f_{\widetilde{N}}\| \leq C\widetilde{N}^{-\frac{m}{d}}\|f\|_{H^m(\Omega)},$$

where d is the dimension,$\widetilde{N}$ is the total degree of freedom, $H^m(\Omega)$ is the Sobolev space.

Approximation estimate of deep learning

东方理工高等研究院
EASTERN INSTITUTE FOR ADVANCED STUDY
宁波东方理工大学(暂名)

- A priori error estimates

$$E\left[\left(f(x)-f(x;\theta)\right)^2\right] \leq Cn^{-\alpha}\|f\|_Y.$$

- **No Curse of Dimensionality! Here α is independent of the dimension d.**

Approximation estimate of the two-layer neural network in Barron space

东方理工高等研究院
EASTERN INSTITUTE FOR ADVANCED STUDY
南方科技大学(暂名)

- There exists a NN $f_n(x;\theta)$ such that [Weinan E, Chao Ma, and Lei Wu, *Constructive Approximation, 2021*]

$$E\left[\left(f(x)-f_n(x;\theta)\right)^2\right] \le \frac{C\|f\|_B^2}{n},$$

where $\|\cdot\|_B$ is the Barron norm, n is the size of the neural network, C is a constant independent of d, and

$$f_n(x;\theta) = \frac{1}{n}\sum_{j=1}^{n} a_j \sigma(\boldsymbol{b}_j^T \boldsymbol{x} + c_j)\,.$$

Contents

- Introduction
  - Classical numerical methods
  - Deep neural networks
- Solving PDEs using deep learning
  - PINNs
  - Deep Ritz method
  - Approximation theory
- Numerical examples
  - Some simple examples
  - Adaptive variants
  - 3D shock waves

Poisson equation: 1D

- Consider the 1D Poisson equation with homogeneous boundary conditions

$$\begin{cases} -u''(x) = f(x), x \in (-1,1), \\ u(-1) = u(1) = 0. \end{cases}$$

- Let $f(x) = \pi^2 \sin(\pi x)$, then

$$u(x) = \sin(\pi x).$$

```
def fnn(self, X, W, b):
    A = 2.0*(X - self.Xmin)/(self.Xmax - self.Xmin) - 1.0
    L = len(W)
    for i in range(L-1):
        A = tf.tanh(tf.add(tf.matmul(A, W[i]), b[i]))
    u = tf.add(tf.matmul(A, W[-1]), b[-1])

    return u

def pdenn(self, x, W, b):
    u = self.fnn(x, W, b)

    u_x = tf.gradients(u, x)[0]
    u_xx = tf.gradients(u_x, x)[0]

    rhf = np.pi**2 * tf.sin(np.pi*x)

    f = -u_xx - rhf

    return f
```

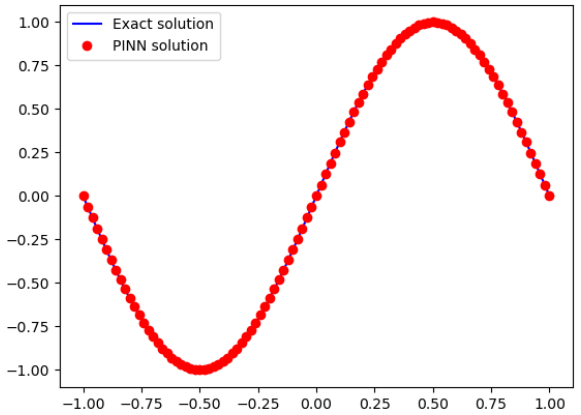
Poisson equation: 1D

- Loss function

$$Loss = \frac{\omega_1}{N_{res}} \sum_{k=1}^{N_{res}} \left(u''_{NN}(x_k) + f(x_k)\right)^2 + \omega_2 (u_{NN}(-1)^2 + u_{NN}(1)^2)$$

```
loss = tf.reduce_mean(tf.square(f_pred)) + \
       1.0* (tf.reduce_mean(tf.square(u_b0_pred)) + \
       tf.reduce_mean(tf.square(u_b1_pred)) )
```

- Case 1: $\omega_1 = \omega_2 = 1$,
- Case 2: $\omega_1 = 1, \omega_2 = 100$.



Test error $\sim 10^{-4}$

Enforce the Boundary Conditions: Method 2

- Let $\bar{u}_{NN}(x) = (x^2 - 1) * u_{NN}(x)$, then

$$\bar{u}_{NN}(-1) = \bar{u}_{NN}(1) = 0.$$

- Loss function

$$Loss = \frac{1}{N_{res}} \sum_{k=1}^{N_{res}} \left(\bar{u}''_{NN}(x_k) + f(x_k)\right)^2$$

```
def fnn_BC2(self, X, W, b):
    Xmax = self.Xmax
    Xmin = self.Xmin
    u = self.fnn(X, W, b)

    u = (X-Xmax)*(X-Xmin) * u

    return u

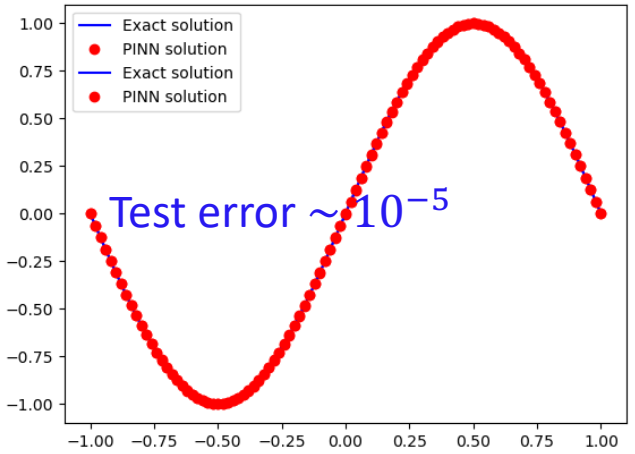
def pdenn_BC2(self, x, W, b):
    u = self.fnn_BC2(x, W, b)

    u_x = tf.gradients(u, x)[0]
    u_xx = tf.gradients(u_x, x)[0]

    rhf = np.pi**2 * tf.sin(np.pi*x)

    f = -u_xx - rhf

    return f
```



Contents

- Introduction
  - Classical numerical methods
  - Deep neural networks
- Solving PDEs using deep learning
  - PINNs
  - Deep Ritz method
  - Approximation theory
- Numerical examples
  - Some simple examples
  - Adaptive variants
  - 3D shock waves

Sharp solution problems

$$L = \min_{\theta}(R^2 + |u_{bc}^* - u_{bc}|^2 + |u_{ic}^* - u_{ic}|^2)$$

where $$\mathcal{L}_f(\boldsymbol{\theta}; \mathcal{T}_f) = \frac{1}{|\mathcal{T}_f|} \sum_{\mathbf{x} \in \mathcal{T}_f} \left\| f\left(\mathbf{x}; \frac{\partial \hat{u}}{\partial x_1}, \ldots, \frac{\partial \hat{u}}{\partial x_d}; \frac{\partial^2 \hat{u}}{\partial x_1 \partial x_1}, \ldots, \frac{\partial^2 \hat{u}}{\partial x_1 \partial x_d}; \ldots; \boldsymbol{\lambda}\right)\right\|_2^2,$$

$$R(\boldsymbol{x}, t) = 0$$

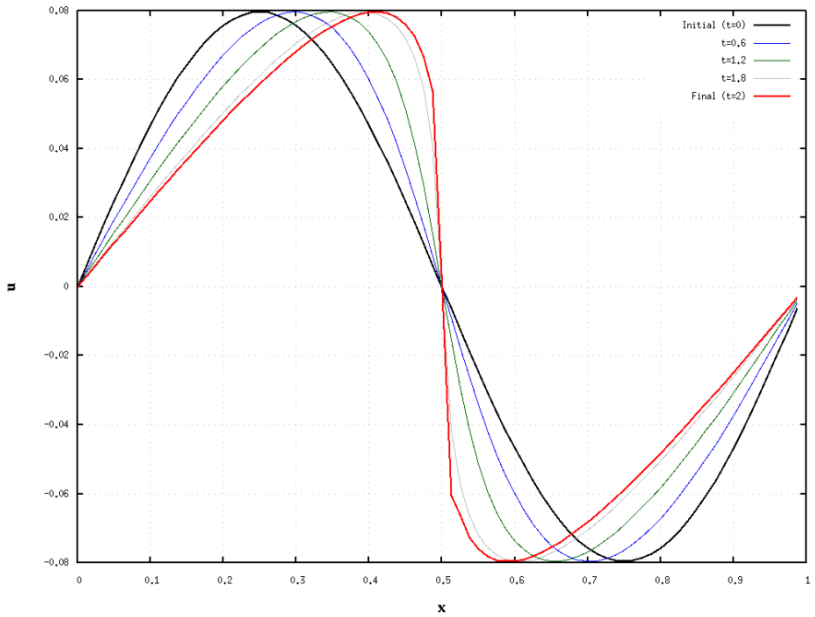


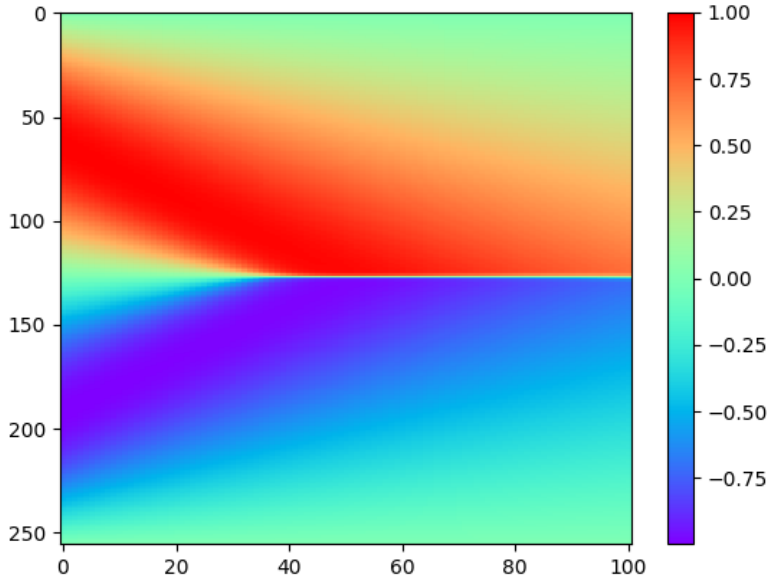
Forward problems: Burgers equation

- Consider the following Burgers equation

$$u_t + uu_x = \frac{0.01}{\pi} u_{xx}, x \in (-1,1), t \in (0,T],$$
$$u(x,0) = u_0(x), u(-1,t) = u(1,t) = 0.$$

- Let $u_0(x) = -\sin(\pi x)$



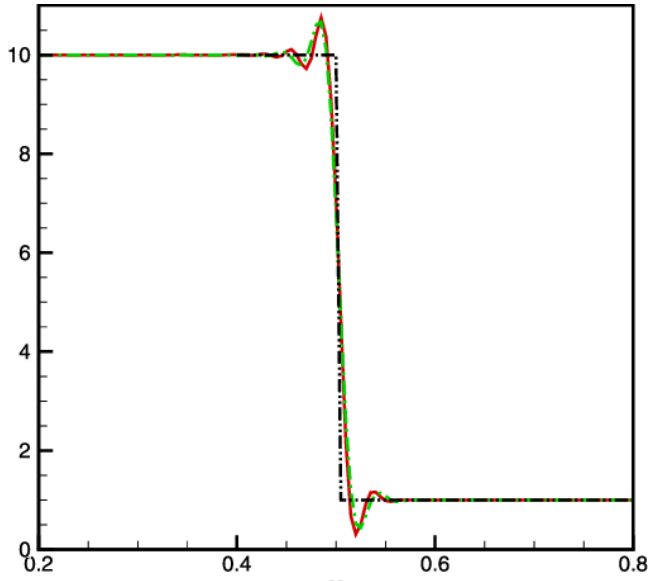


Forward problems: Burgers equation

- Consider the following Burgers equation

$$u_t + uu_x = \frac{0.01}{\pi} u_{xx}, x \in (-1,1), t \in (0,T],$$
$$u(x,0) = u_0(x), u(-1,t) = u(1,t) = 0.$$

- Let $u_0(x) = -\sin(\pi x)$



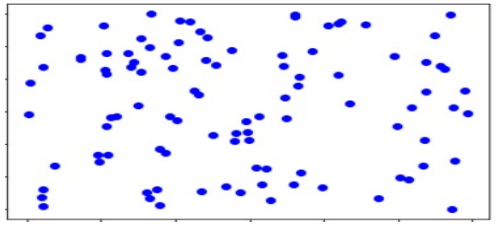
Residual-based adaptive sampling

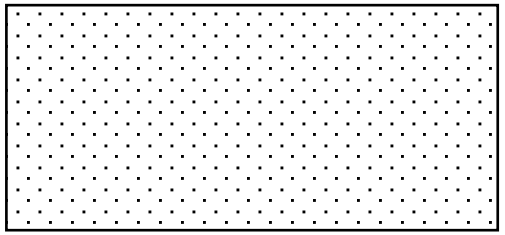
东方理工高等研究院
EASTERN INSTITUTE FOR ADVANCED STUDY
宁波东方理工大学(暂名)

$$L = \min_{\theta}(R^2 + |u_{bc}^* - u_{bc}|^2 + |u_{ic}^* - u_{ic}|^2)$$

$$R(\boldsymbol{x}, t) = 0$$

Step 1.





Step 2. Randomly select # testing points $R(\boldsymbol{x},t) = 0$?

Step 3. Add new training data at locations $R(\boldsymbol{x},t) > 0$

Lu, et al, SIREV, 2021

Residual-based adaptive sampling

东方理工高等研究院
EASTERN INSTITUTE FOR ADVANCED STUDY
宁波东方理工大学(暂名)

Algorithm 1: ASM I: Residual only-based adaptive sampling method

Step 1 Train the PINNs based on the residual point set $\mathcal{T}$;

Step 2 Divide the computational domain into $\boldsymbol{S}$ subdomains $\Omega_i, i = 1, \ldots, \boldsymbol{S}$, and then compute the mean value $(\mathcal{E}_i, i = 1, \ldots, \boldsymbol{S})$ of the absolute value of the residual (R), namely,

$$\mathcal{E}_i = \int_{\Omega_i} |R| d\boldsymbol{x},$$

using the Gaussian quadrature method for each sub-domain (See Fig. 2 for a 1D representative case).

Step 3 Let $\mathcal{E}_{max}$ denote the largest mean value of the PDE residuals for all subdomains, i.e., $\mathcal{E}_{max} = \max_{i=1,\ldots \boldsymbol{S}}\{\mathcal{E}_i\}$. If $\mathcal{E}_{max} < \mathcal{E}_c$, iteration stops ($\mathcal{E}_c$ is a user-defined convergence criterion). Otherwise, randomly sample $\mathcal{M}$ data in the sub-domain which has the largest $\mathcal{E}$, and then compute the residuals for the selected points. Add m_R points which have the largest absolute value of the residuals in the residual point set $\mathcal{T}$.

Step 4 Repeat Step 1 - 3.

Forward problems: Burgers equation

东方理工高等研究院
EASTERN INSTITUTE FOR ADVANCED STUDY
宁波东方理工大学(暂名)

- Consider the following Burgers equation

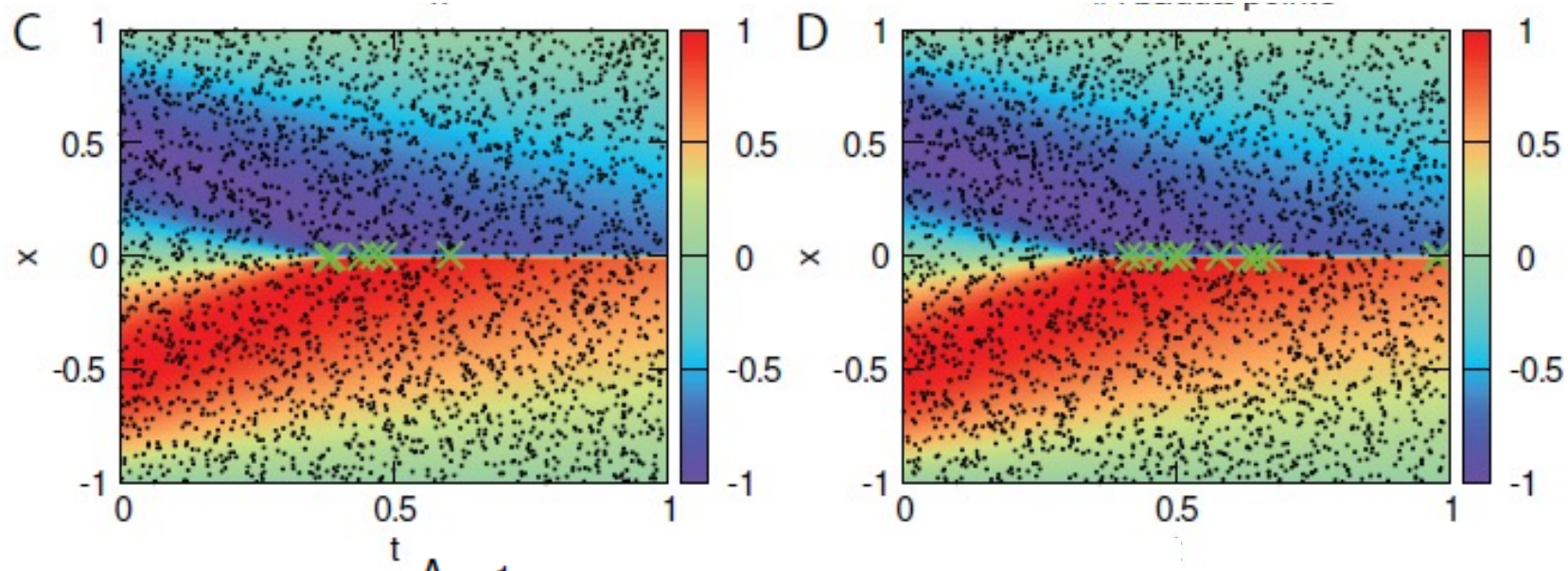
$$u_t + uu_x = \frac{0.01}{\pi} u_{xx}, x \in (-1,1), t \in (0,T],$$
$$u(x,0) = u_0(x), u(-1,t) = u(1,t) = 0.$$

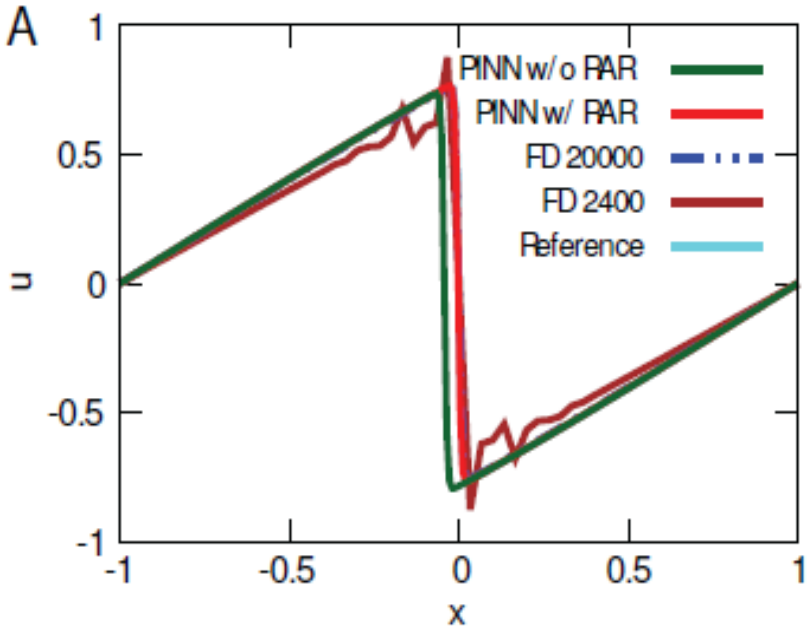
- Let $u_0(x) = -\sin(\pi x)$
- Randomly select 2500 residual points

- # of testing points for $R(\boldsymbol{x},t)$ – 100,000

Lu, et al, SIREV, 2021

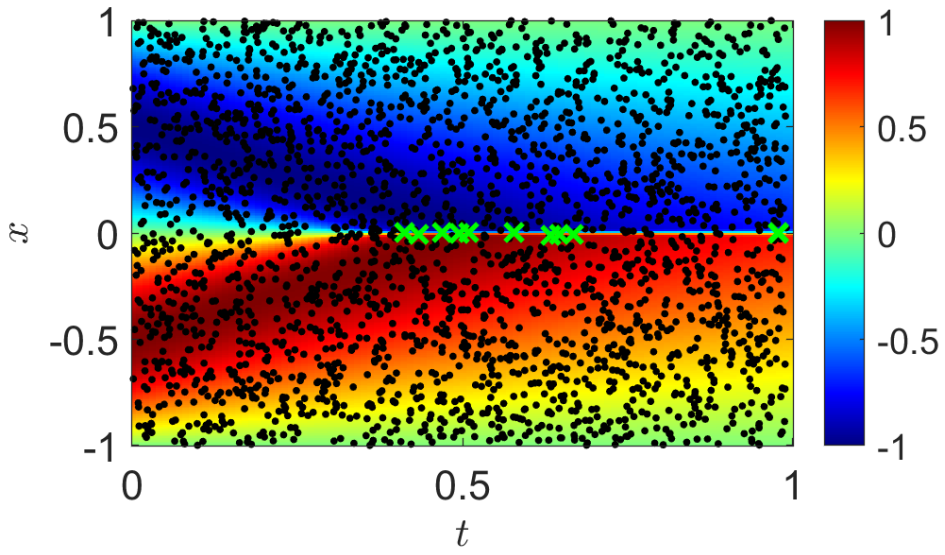
Residual-based adaptive sampling

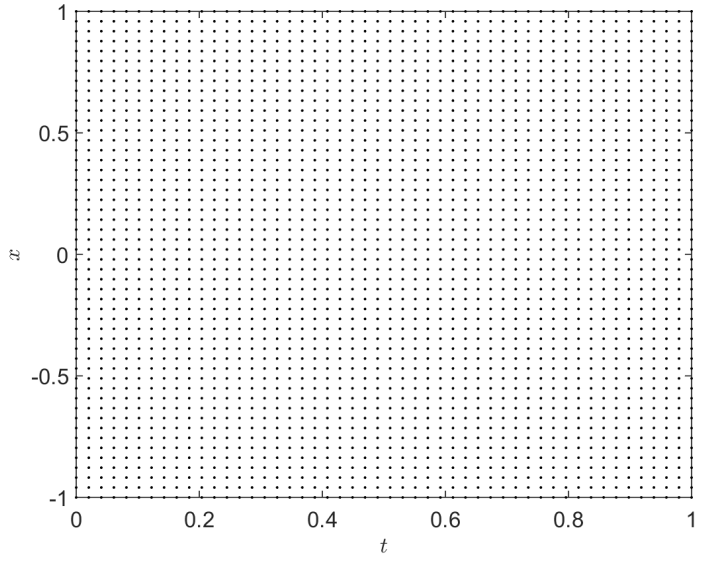




Lu, et al, SIREV, 2021

Residual-based adaptive sampling





Lu, et al, SIREV, 2021

Residual-based adaptive sampling

- Easy to implement
- Computationally efficient
- Effective for both spatial and temporal domain

Forward problems: 1D Euler equation

东方理工高等研究院
EASTERN INSTITUTE FOR ADVANCED STUDY
宁波东方理工大学(暂名)

$$\partial_t \boldsymbol{U} + \partial_x \boldsymbol{f} = 0, \ x \in [0,1], \ t \in [0,2],$$

where

$$\boldsymbol{U} = \begin{cases} \rho \\ \rho u \\ E \end{cases}, \text{ and } \boldsymbol{f} = \begin{cases} \rho u \\ p + \rho u \\ u(p+E) \end{cases},$$

$$p = (\gamma - 1)\left(E - \frac{1}{2}\rho u^2\right) \qquad \gamma = 1.4$$

$x = 0.5$ $\quad (\rho_L, u_L, p_L) = (1.4, 0.1, 1.0), \ \ (\rho_R, u_R, p_R) = (1.0, 0.1, 1.0)$

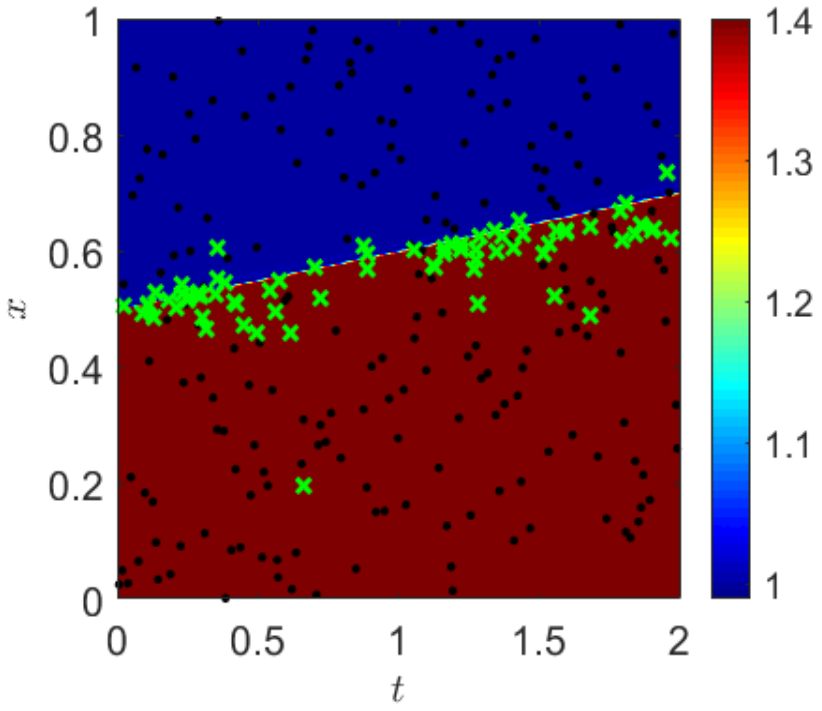
$$\rho(x,t) = \begin{cases} 1.4, & x < 0.5 + 0.1t, \\ 1.0, & x > 0.5 + 0.1t, \end{cases} \quad u(x,t) = 0.1, \ p(x,t) = 1.0.$$

Mao & Meng, AMM, 2023

Forward problems: 1D Euler equation

东方理工高等研究院
EASTERN INSTITUTE FOR ADVANCED STUDY
宁波东方理工大学(暂名)

- Randomly select 200 residual points
- $(10\times10),(10\times10)$ # of subdomains: $x\times t$, # of quadrature points: $x\times t$



Mao & Meng, AMM, 2023

Residual/gradient-based adaptive sampling

Algorithm 2: ASM II: Residual/gradient-based adaptive sampling method

Step 1 Train the PINNs based on the residual point $\mathcal{T}$;

Step 2 Divide the computational domain into $\boldsymbol{S}$ subdomains $\Omega_i, i = 1, \ldots, \boldsymbol{S}$, and then compute the mean value $(\mathcal{E}_i, i = 1, \ldots, \boldsymbol{S})$ of the absolute value of the residual (R) and the mean value $(\mathbf{E}_i, i = 1, \ldots, \boldsymbol{S})$ of the absolute value of the gradient of the solution (∇u), namely,

$$\mathcal{E}_i = \int_{\Omega_i} |R| d\boldsymbol{x}, \quad \mathbf{E}_i = \int_{\Omega_i} |\nabla u| d\boldsymbol{x}$$

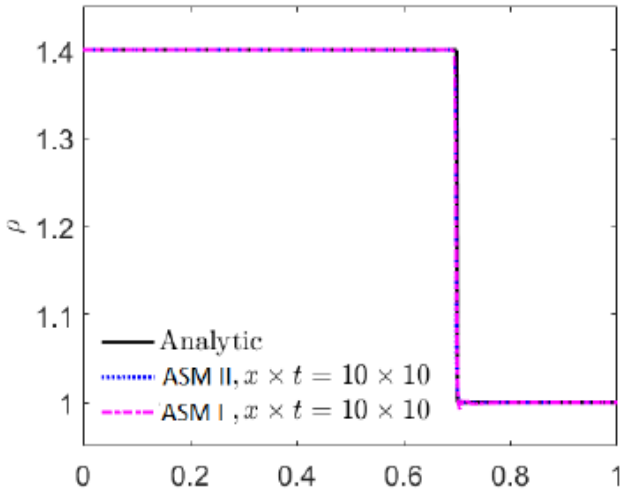
using the Gaussian quadrature method for each sub-domain;

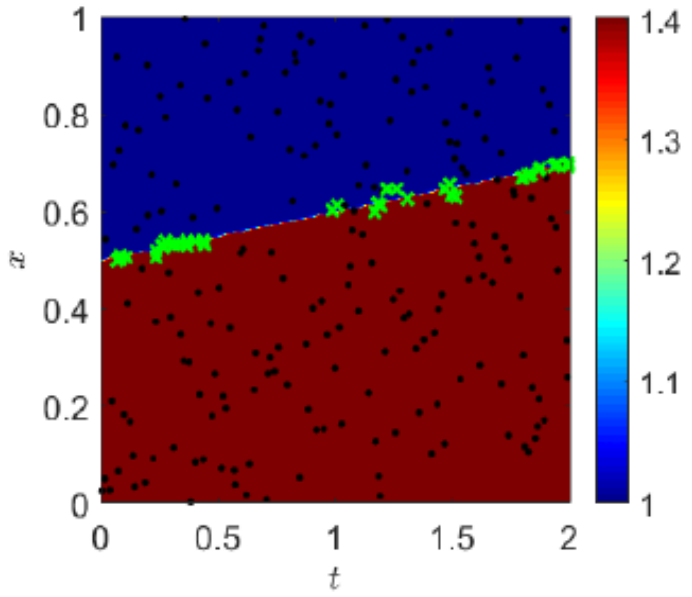
Step 3 Let $\mathcal{E}_{max} = \max_{i=1,\ldots \boldsymbol{S}}\{\mathcal{E}_i\}$ and $\mathbf{E}_{max} = \max_{i=1,\ldots \boldsymbol{S}}\{\mathbf{E}_i\}$. If $\mathcal{E}_{max} < \mathcal{E}_c$, iteration stops. Otherwise, randomly sample $\mathcal{M}$ data in the sub-domain which has the largest $\mathcal{E}$ and $\mathcal{M}$ data in the sub-domain which has the largest $\mathbf{E}$, and then compute the residuals and gradients for selected points, respectively. Add m_R points which have the largest $|R|$ and $m_{\nabla u}$ points which have the largest $|\nabla u|$ in the residual point set $\mathcal{T}$.

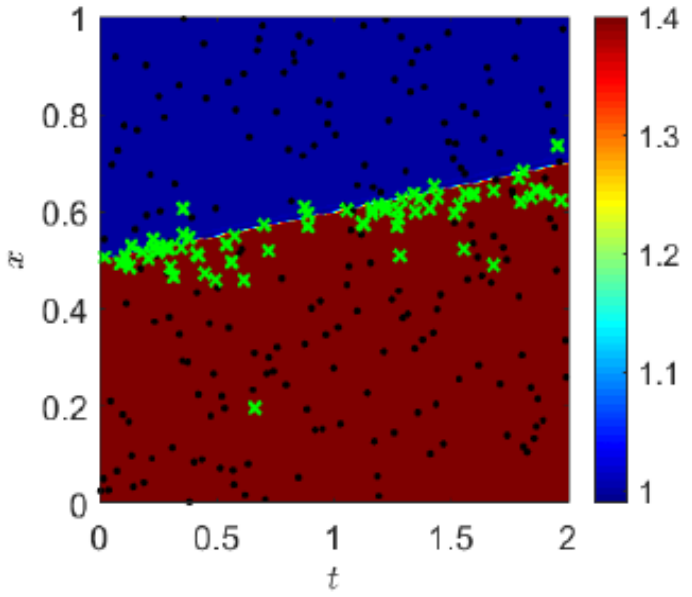
Step 4 Repeat Step 1 - 3.

Mao & Meng, AMM, 2023

Forward problems: 1D Euler equation







Mao & Meng, AMM, 2023

Forward problems: 9-dimensional Poisson equation

东方理工高等研究院
EASTERN INSTITUTE FOR ADVANCED STUDY
宁波东方理工大学(暂名)

$$-\Delta u(\boldsymbol{x}) = f(\boldsymbol{x}), \quad \boldsymbol{x} \in \Omega := [-1,1]^d,$$
$$u(\boldsymbol{x}) = 0, \quad \boldsymbol{x} \in \partial\Omega.$$

$$u(\boldsymbol{x}) = e^{-10\|\boldsymbol{x}\|^2} \quad d = 9$$

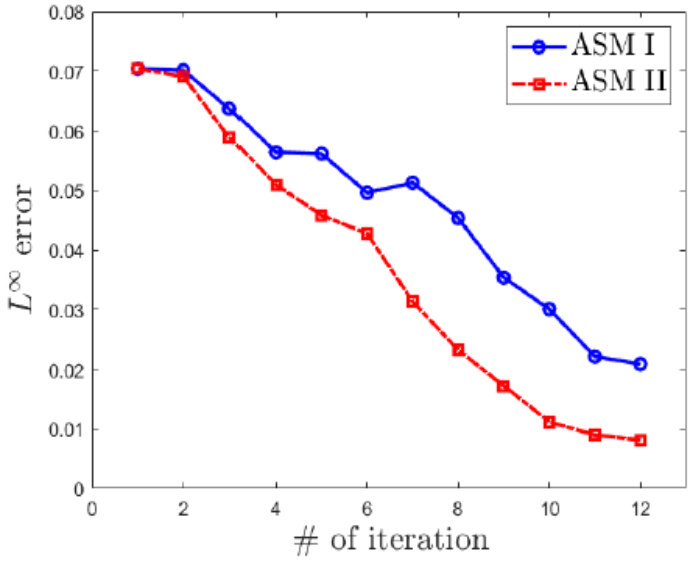


Figure 9: Convergence of the L^∞ error for the high dimensional Poisson problem using ASM I and ASM II. $m_R = 500$ for ASM I while $m_R = 300$, $m_{\nabla u} = 200$ for ASM II.

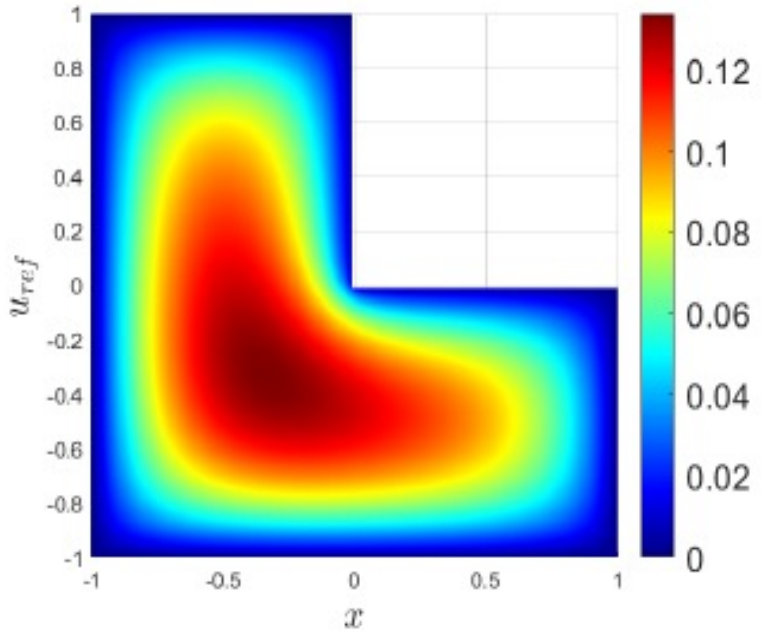
Mao & Meng, AMM, 2023

Forward problems: Complex geometry

东方理工高等研究院
EASTERN INSTITUTE FOR ADVANCED STUDY
宁波东方理工大学(暂名)

$$
\begin{aligned}
-\Delta u(x,y) &= 1, \quad (x,y) \in \Omega, \\
u(x,y) &= 0, \quad (x,y) \in \partial\Omega.
\end{aligned}
$$

$x, y \in [-1,1]$



Mao & Meng, AMM, 2023

Forward problems: Complicated Boundaries

Algorithm 3: ASM III: Residual/gradient/BC-based adaptive sampling method

Step 1 Train the PINNs based on the residual point set $\mathcal{T}$ and the boundary point set $\mathcal{B}$;

Step 2 Divide the computational domain into $\boldsymbol{S}$ subdomains $\Omega_i, i = 1, \ldots, \boldsymbol{S}$, and compute

$$\mathcal{E}_i = \int_{\Omega_i} |R| d\boldsymbol{x}, \quad \mathbf{E}_i = \int_{\Omega_i} |\nabla u| d\boldsymbol{x}$$

using the Gaussian quadrature method for each sub-domain; using the Gaussian quadrature method for each sub-domain;

Step 3 Let $\mathcal{E}_{max} = \max_{i=1,\ldots \boldsymbol{S}}\{\mathcal{E}_i\}$ and $\mathbf{E}_{max} = \max_{i=1,\ldots \boldsymbol{S}}\{\mathbf{E}_i\}$. If $\mathcal{E}_{max} < \mathcal{E}_c$, iteration stops. Otherwise, randomly sample $\mathcal{M}$ data in the sub-domain which has the largest $\mathcal{E}$ and $\mathcal{M}$ data in the sub-domain which has the largest $\mathbf{E}$, and then compute the corresponding residuals and gradients for selected points, respectively. Add m_R points which have the largest $|R|$ and $m_{\nabla u}$ points which have the largest $|\nabla u|$ in the residual point set .

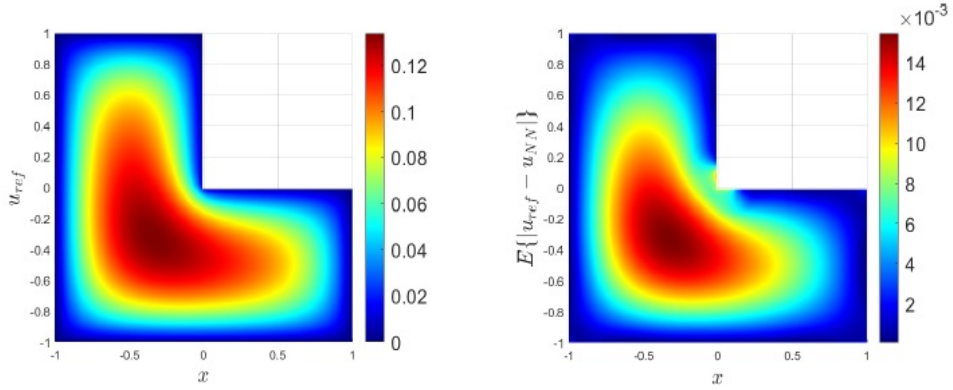
Step 4 Randomly sample $\mathcal{M}_{BC}$ boundary points and compute the absolute value of u at these boundary points. Then add m_{BC} points which have the largest $|u|$ in the boundary point set $\mathcal{B}$.

Step 5 Repeat Step 1 - 4.

Mao & Meng, AMM, 2023

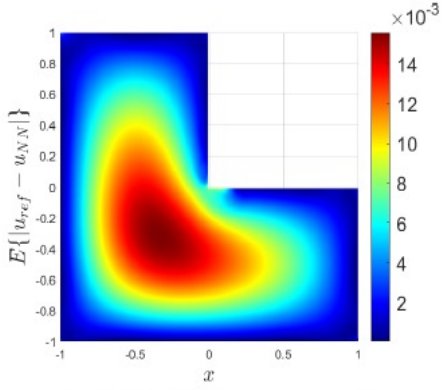
Forward problems: Complicated Boundaries

- Randomly select 400 residual points
- $(25\times25),(12\times12)$

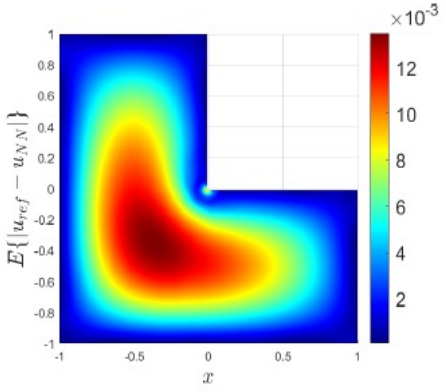


(a) Reference solution

(b) ASM I: $E\{|u_{ref}-u_{NN}|\}$



(c) ASM II: $E\{|u_{ref}-u_{NN}|\}$



(d) ASM III: $E\{|u_{ref}-u_{NN}|\}$

Mao & Meng, AMM, 2023

Forward problems: Complicated Boundaries

- Randomly select 400 residual points
- (25×25),(12×12)

	ASM I	ASM II	ASM III (for BCs)
Average # of Added points	57	60	20
Relative L^2 error	$(10.57 \pm 1.92)\%$	$(10.39 \pm 1.12)\%$	$(8.60 \pm 0.91)\%$
Relative L^∞ error	$(25.38 \pm 14.44)\%$	$(14.48 \pm 3.08)\%$	$(9.80 \pm 1.01)\%$

Table 3: Results for the Poisson problem over the L-shape domain. $m_R = 2$ for ASM I, $m_R = m_{\nabla u} = 1$ for ASM II, $m_R = m_{\nabla u} = 1, m_{BC} = 2$ for ASM III: Average numbers of added points and relative L^2 and L^∞ errors using ASM I, ASM II and ASM III. We run the code 10 times for each case.

Mao & Meng, AMM, 2023

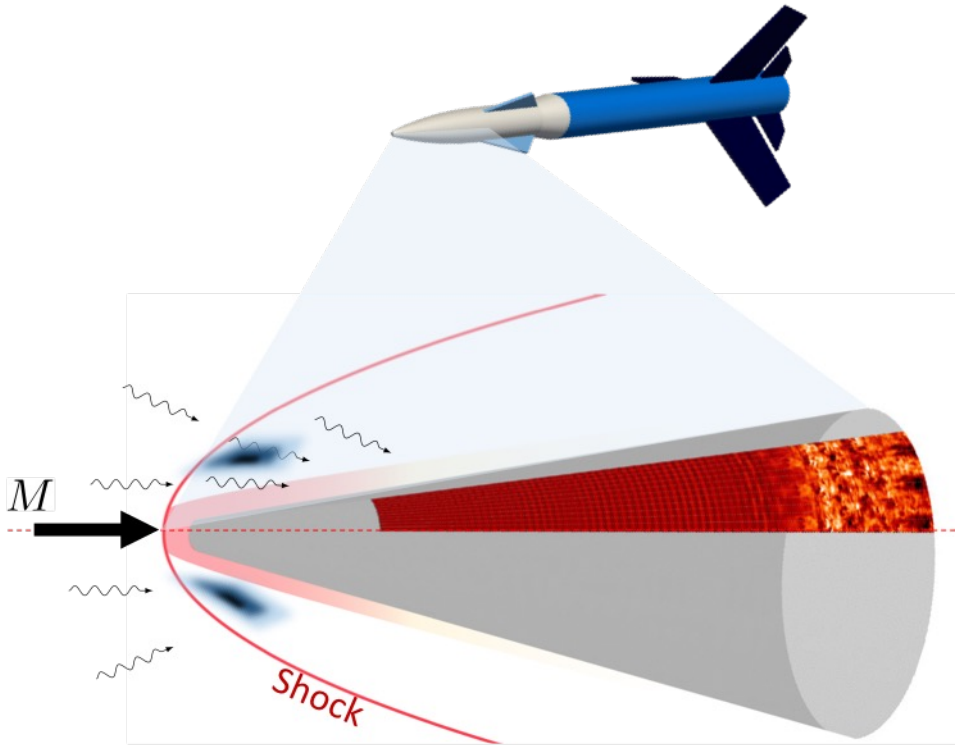
Adaptive Methods for PINNs

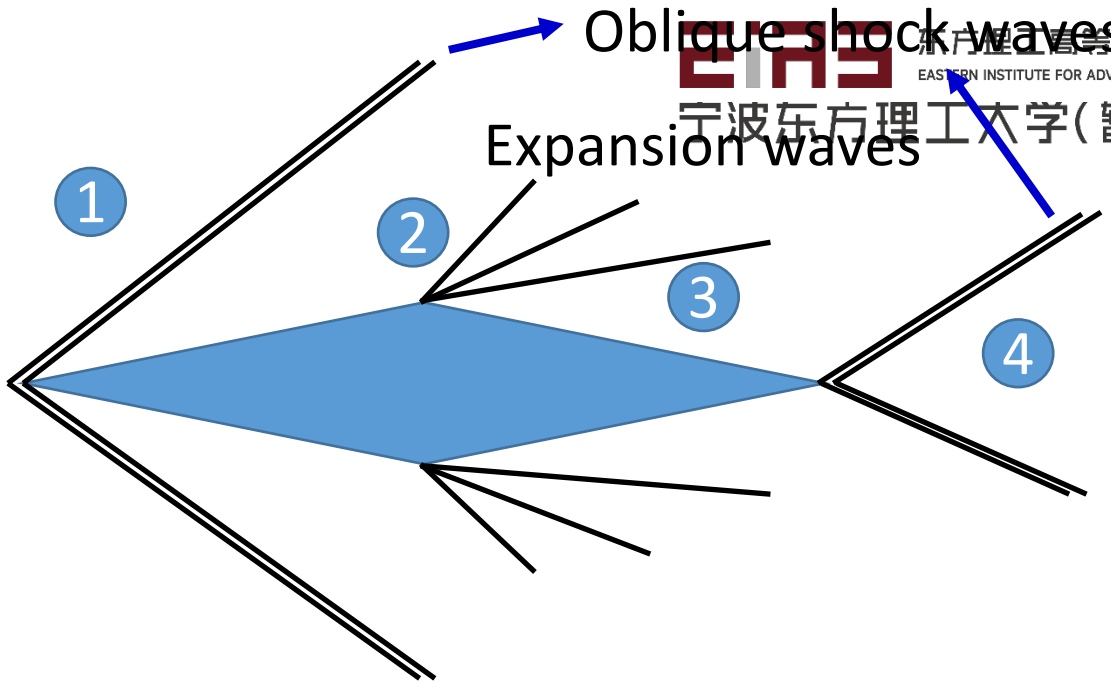
- Residual/gradient-based Adaptive Sampling Method (Lu-Meng-M-Karniadakis2021, M-Meng2023)
- Using the gradient of residual (Yu-Lu-Meng-Karniadakis, CMAME, 2021)
- Comprehensive study of non-adaptive and residual-based adaptive sampling (Wu-Zhu-Tan-Kartha-Lu, CMAME, 2023)
- Failure-informed adaptive sampling (Gao-Yan-Zhou, arXiv:2210.00279)
- Residual-quantile adjustment (Han-Cai-Wu-Zhou, arXiv:05315.05315)
- Self-adaptive PINN (McClenny-Neto, JCP, 2023)
- Adaptive activation function (Jagtap-Kawaguchi-Karniadakis, JCP, 2020)
- DAS-PINNs (Tang-Wan-Yang, JCP, 2023)
- ……

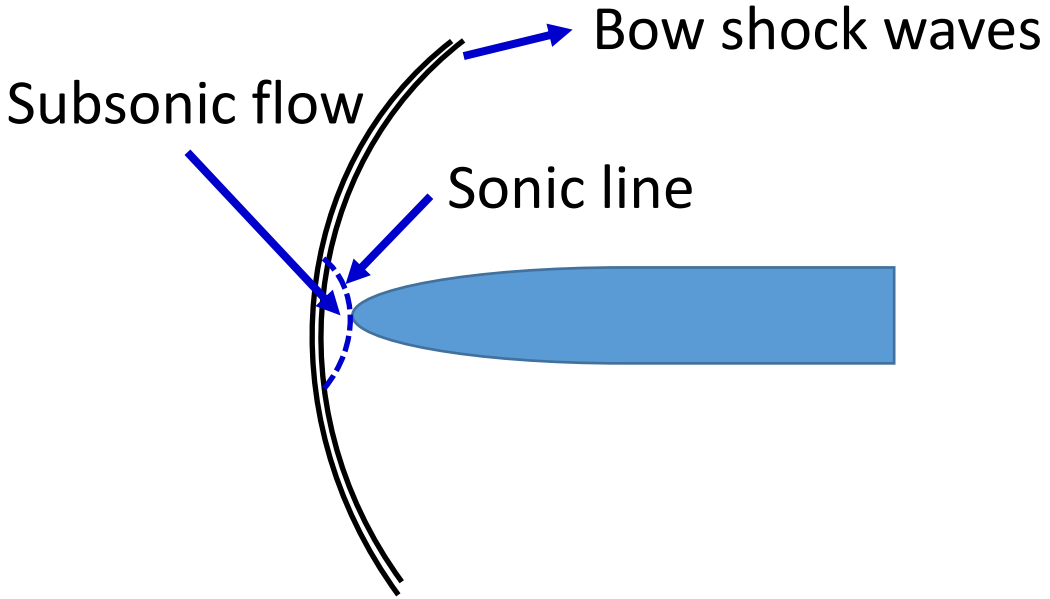
Contents

- Introduction
  - Classical numerical methods
  - Deep neural networks
- Solving PDEs using deep learning
  - PINNs
  - Deep Ritz method
  - Approximation theory
- Numerical examples
  - Some simple examples
  - Adaptive variants
  - 3D shock waves

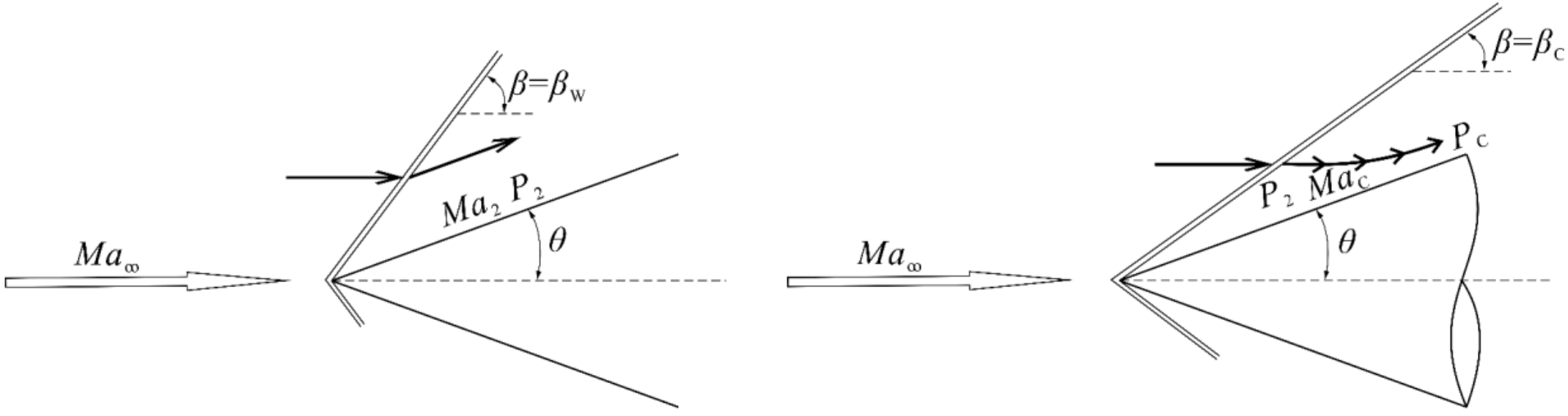
High-speed flows







Supersonic flow over cones (3D cases)

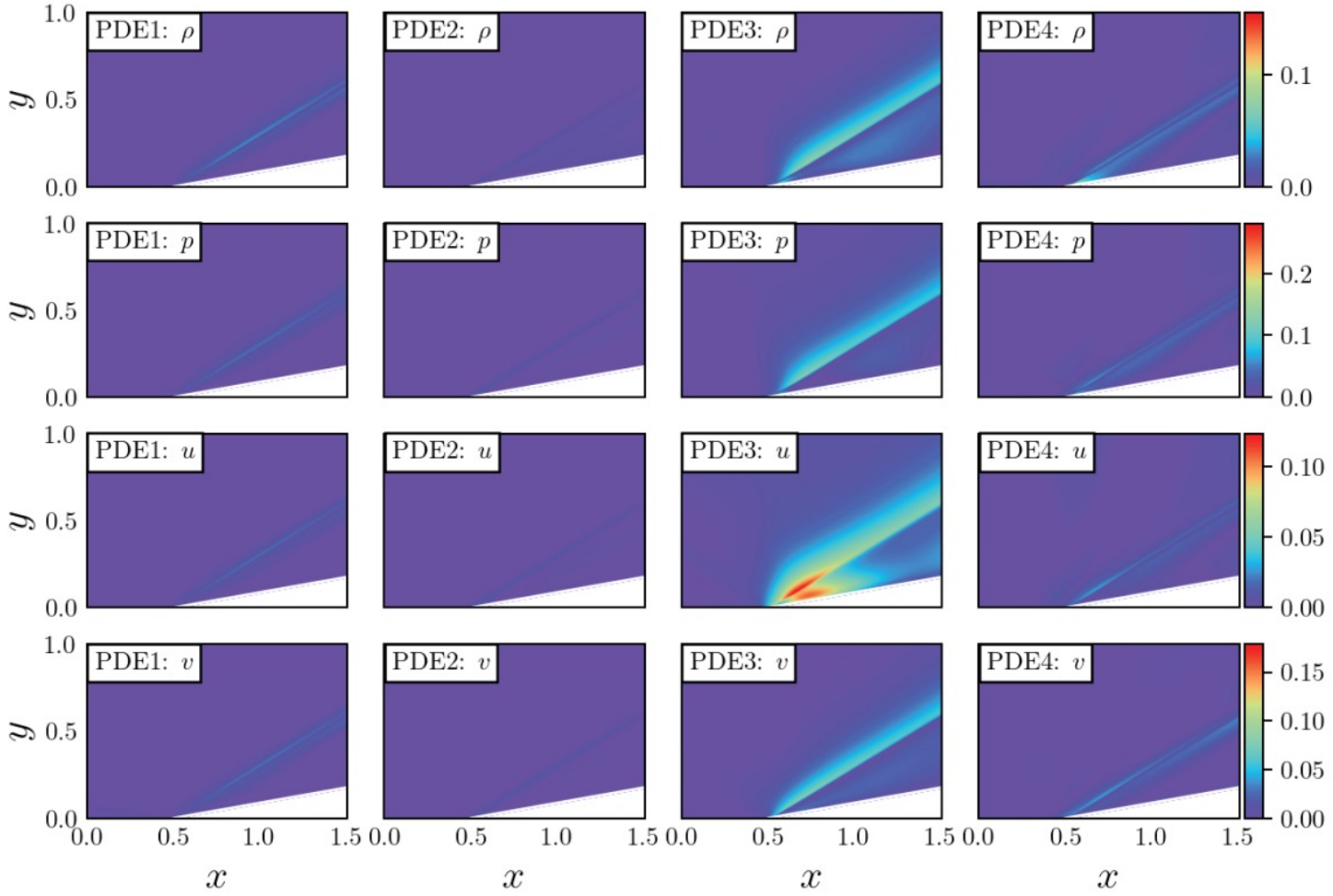


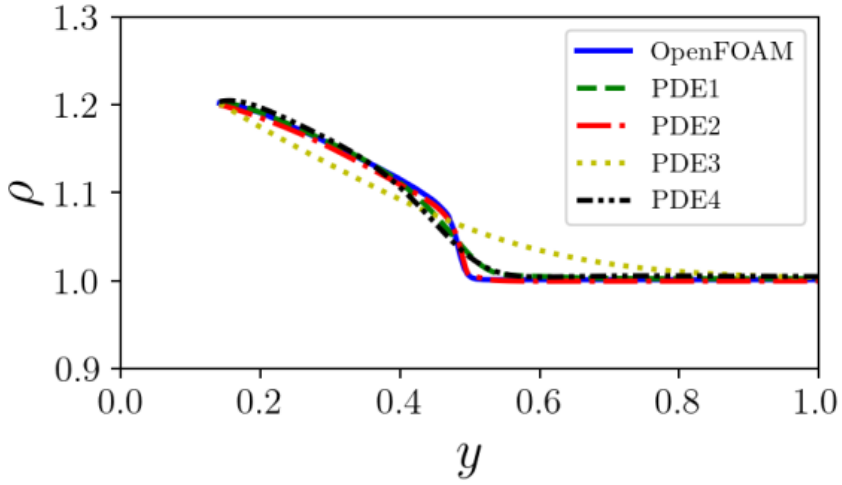
a) Wedge

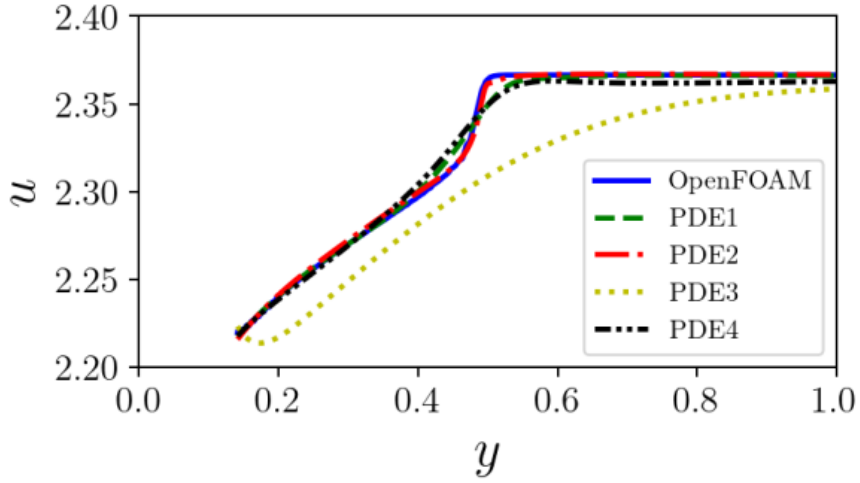
b) Cone

Supersonic flow over cones

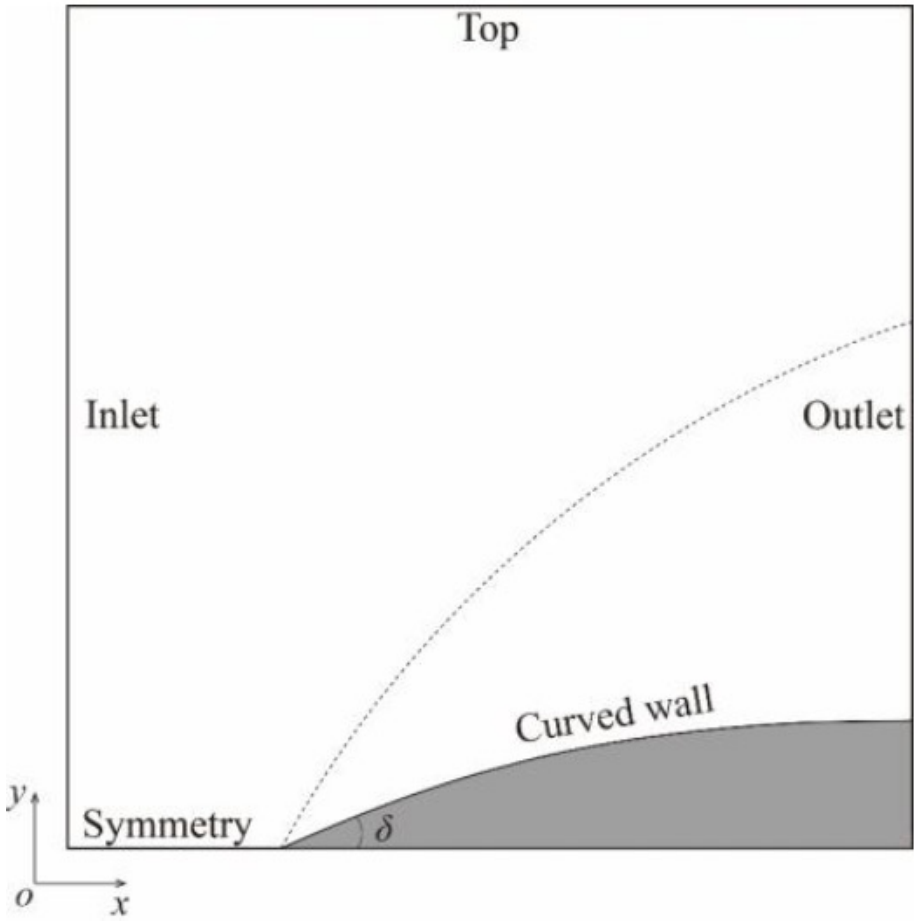
PDE1 and PDE2: Using **specific enthalpy**
PDE3 & PDE4: Using **total energy**



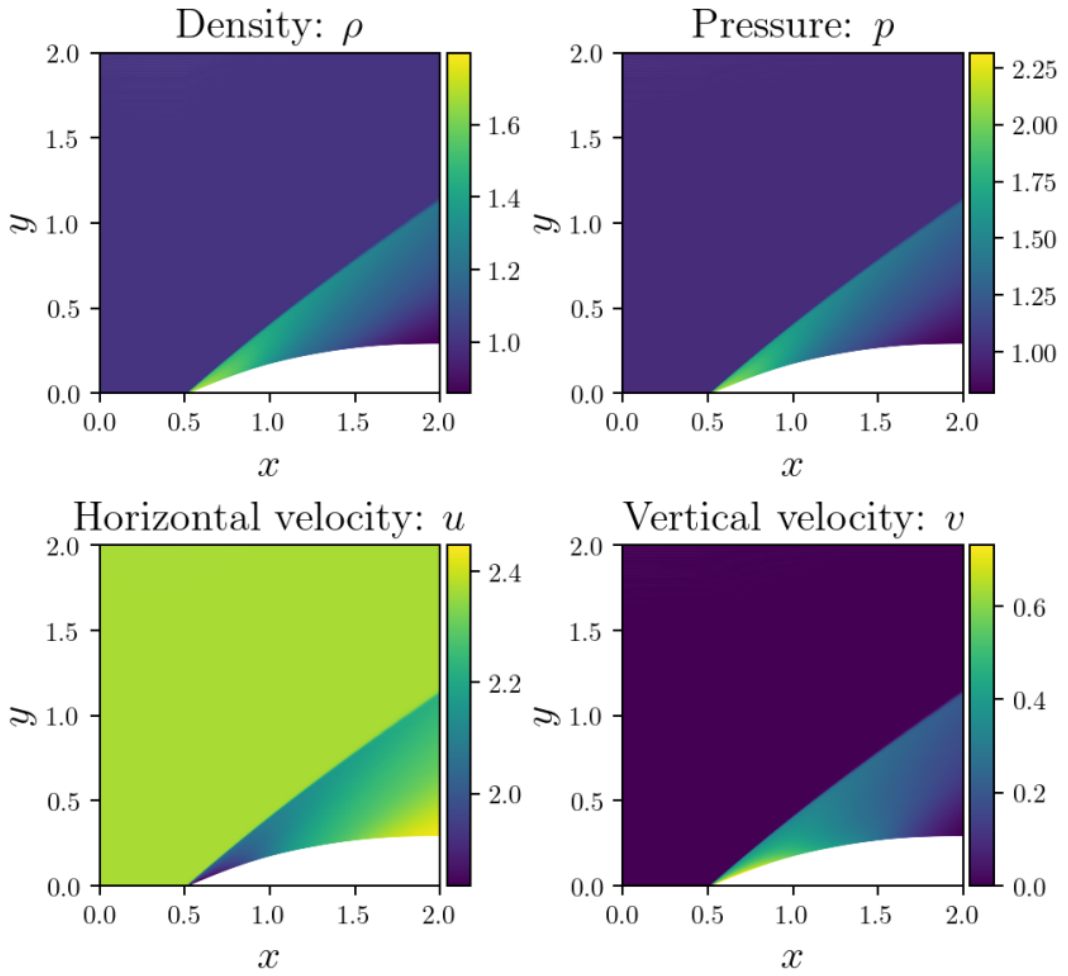




Axisymmetric curved shock wave



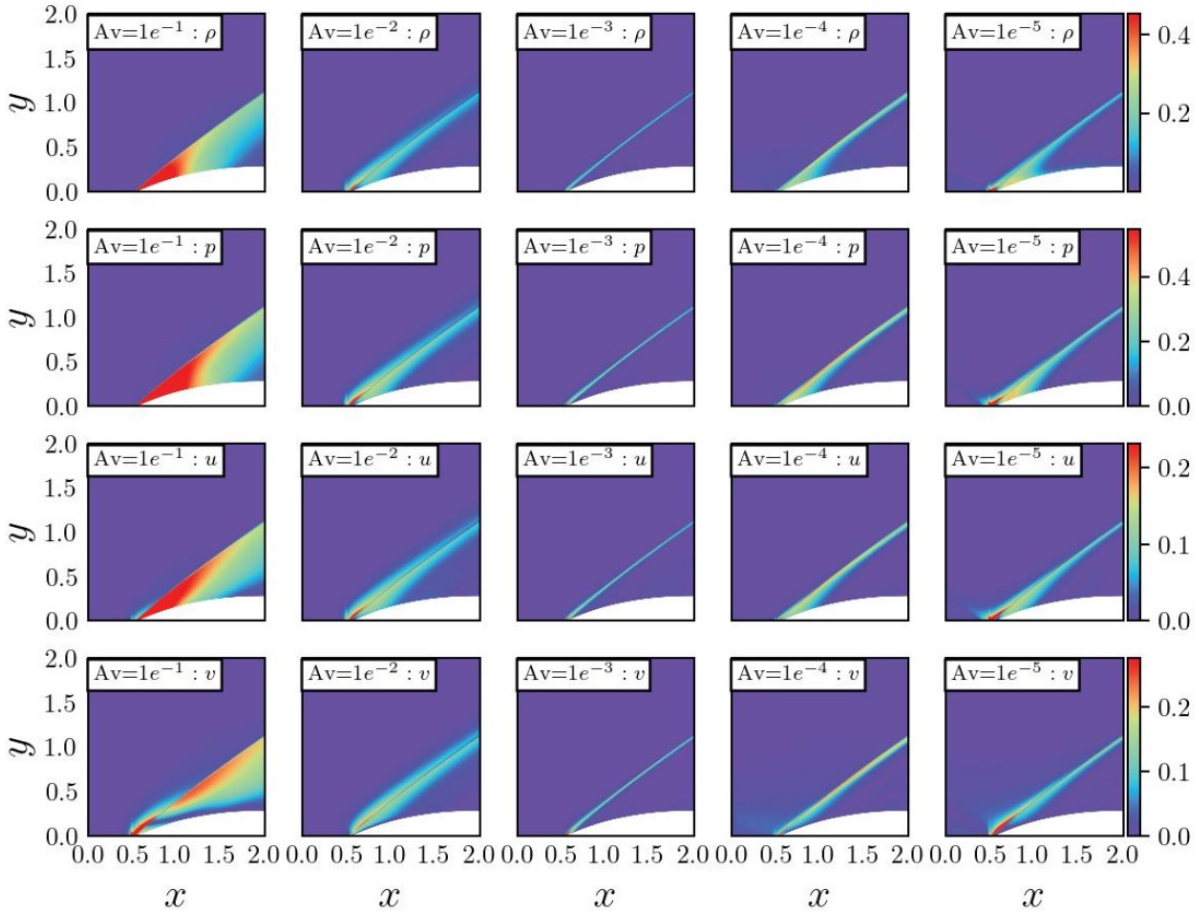
OpenFOAM solution

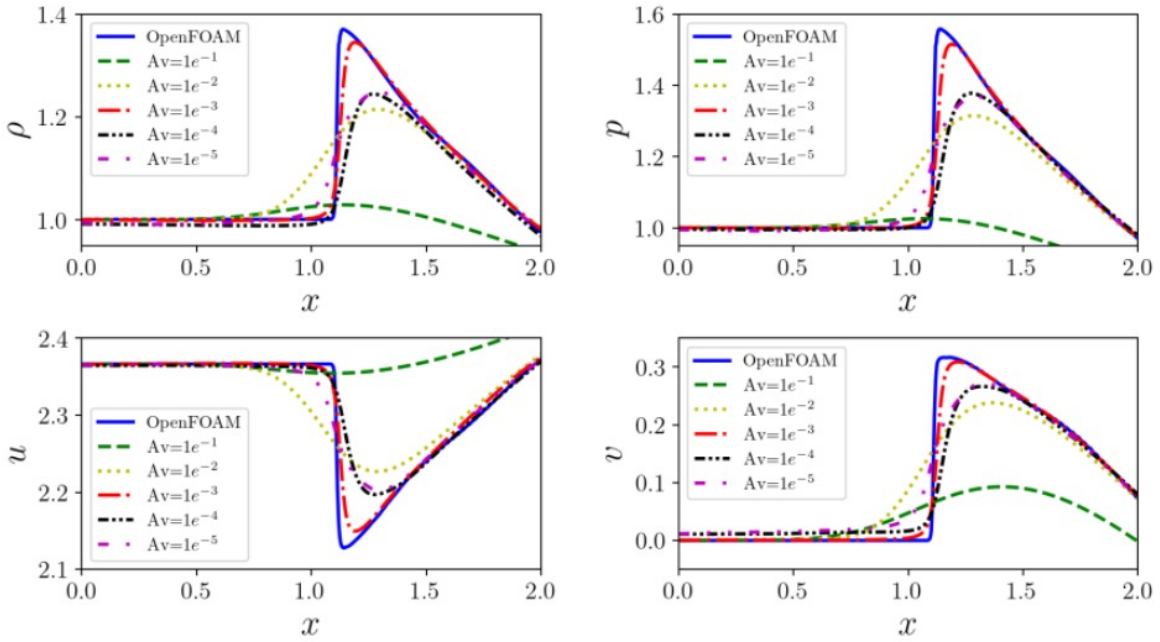


Axisymmetric curved shock wave

东方理工高等研究院
EASTERN INSTITUTE FOR ADVANCED STUDY
宁波东方理工大学(暂名)

Point-wise error

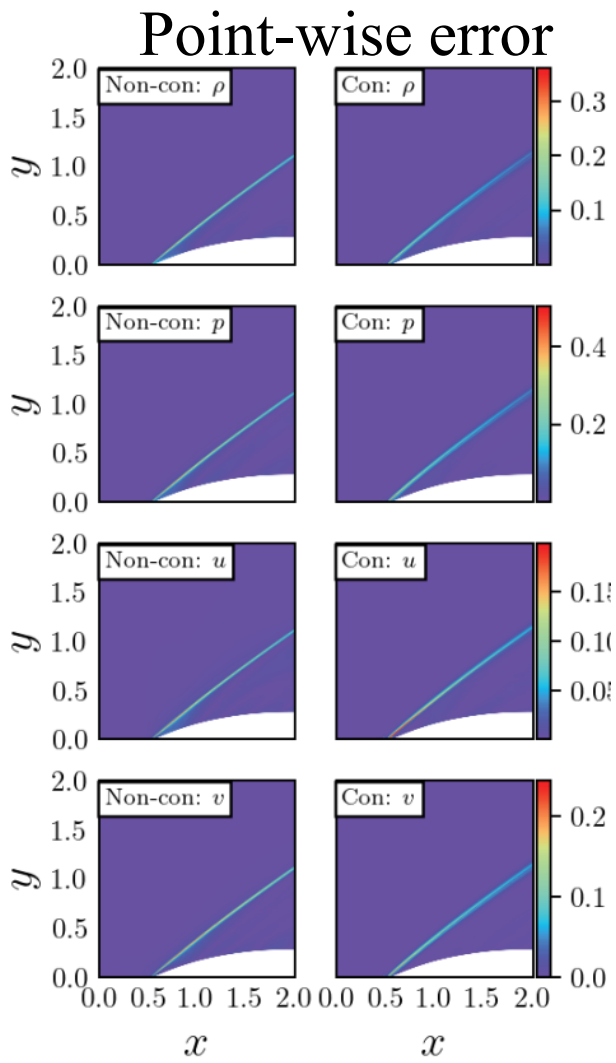


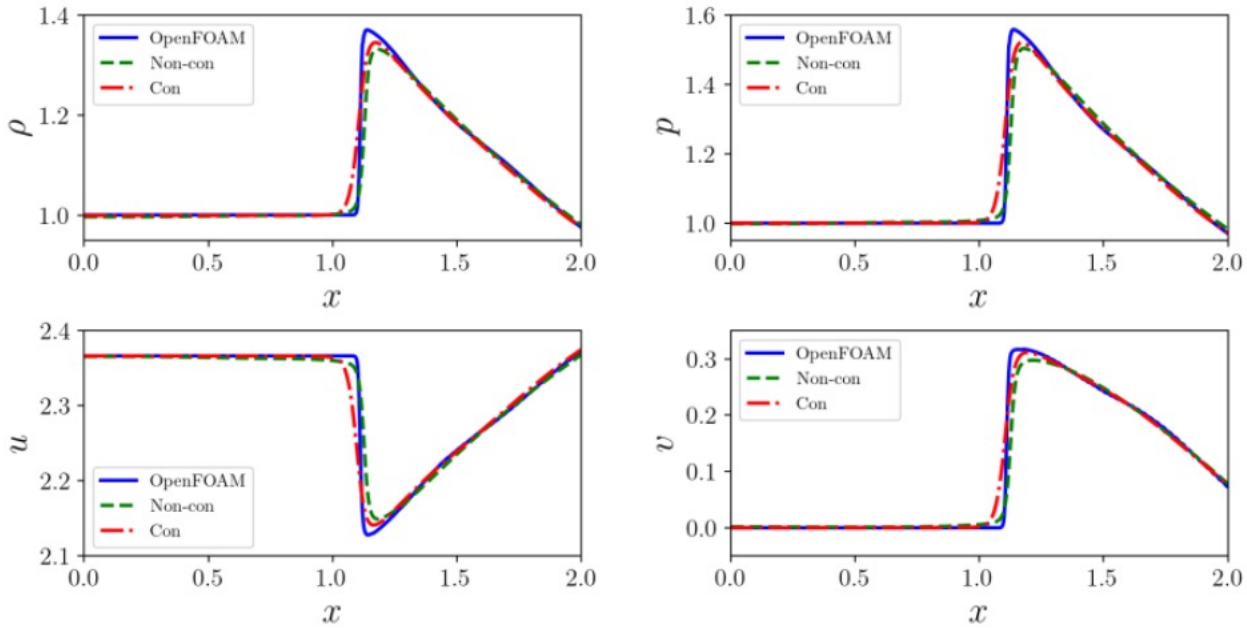


- ✓ Av represents Artificial Viscosity
- ✓ Using **Constant** Artificial Viscosity

Axisymmetric curved shock wave

东方理工高等研究院
EASTERN INSTITUTE FOR ADVANCED STUDY
宁波东方理工大学(暂名)





- ✓ Av represents Artificial Viscosity
- ✓ Using **Learnable** Artificial Viscosity

Thank you for your attention!